OsdagBridge
Project Management Handbook

A start-from-zero guide to the board, epics, and sprints.

If you have never run a project board, never heard the word "epic," and are not sure what a "sprint" has to do with software, this book is written for you. Read it front to back once. After that, keep it next to the board.

Version 1.2 | 1 August 2026

Part of **Osdag Project Management** — one engine, many FOSSEE-project boards.

Repository: `nishikantmandal007/Osdag-Project-Management`

Board: project #3, "OsdagBridge"

Contents

Before you start

Who this is for. New contributors and interns joining OsdagBridge, and anyone who has to keep the project board honest. No project-management background is assumed. Every term is defined the first time it appears.

How the book is laid out.

- **Part I** teaches project management from scratch, with plain analogies. Read this if words like *epic*, *backlog*, or *triage* are new to you.
- **Part II** maps those ideas onto OsdagBridge specifically: our labels, our ten epics, our bug tiers.
- **Part III** walks you through building the board from an empty page, one field and one view at a time.
- **Part IV** is the daily and fortnightly routine: how a sprint runs and how one issue travels from "someone reported it" to "done."
- **Part V** answers three questions people always ask about running a board in the open: can outsiders break it, how do interns join without repository access, and were all those robots really necessary.
- **Part VI** is reference material: a glossary, a map of which configuration file controls what, and a one-page cheat sheet.

Note

This handbook is the friendly, long-form guide. The short checklists it is based on live next to it as `SOP-01` through `SOP-10` and `BOARD-SETUP.md`. When this book says "see the triage SOP," it means one of those files. You never need them to follow along here, but they are the terse reference once you know the ropes.

Part I

Project Management from Zero

CHAPTER 1

Why a project needs a “management layer”

OsdagBridge is real engineering software. It sizes and checks steel bridges. About ten people have written roughly ninety thousand lines of Python over a thousand commits. That is a serious amount of work by a serious group of people.

For a long time the way that work got tracked was a single long list of tickets on GitHub. Ninety-three of them, open at once. No order beyond newest-on-top. No way to tell a wrong-answer bug from a spelling mistake. No way to see who was doing what, or which tickets belonged together, or what had to ship before a release. A majority of them had nobody’s name on them at all.

That list is not a failure. It is what every growing project produces if nothing organises the incoming work. The pile just grows until people stop being able to see the shape of it.

In plain English

A “project management layer” is not extra bureaucracy on top of the real work. It is the difference between a kitchen during a dinner rush with tickets stuck up in order, owned by one cook each, and the same kitchen with orders shouted into the air and everyone hoping someone heard. Same cooks, same food. One of them serves dinner.

The goal of this system is not to make people work faster. OsdagBridge’s throughput was already climbing on its own. The goal is three plainer things:

- **Intake discipline.** Every new report gets sorted the same way, so nothing rots unseen at the bottom of a list.
- **Ownership.** Every piece of work has one name on it, so “someone should look at this” becomes “you are looking at this.”
- **Traceability.** You can follow any result on screen back to the line of code and the test that guards it. For structural software, where a wrong number can be an unsafe bridge, this is the whole point.

CHAPTER 2

The words, with pictures

Project management has its own vocabulary. None of it is hard once you have a picture for it. Here is the whole cast.

2.1 Issue

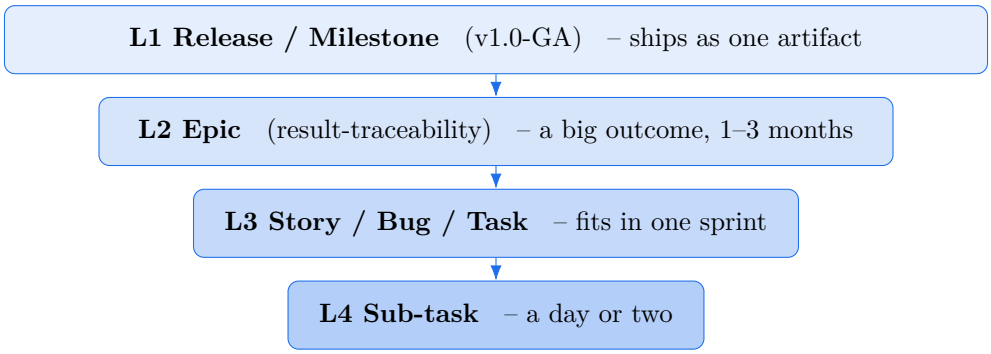
An **issue** is one unit of work or one conversation. A bug report is an issue. A feature request is an issue. "Rename this button" is an issue. On GitHub, an issue has a number (`#42`), a title, a description, labels, and one or more people assigned to it. Everything else in this chapter is a way of grouping, sorting, or scheduling issues.

2.2 The work-item hierarchy: Release, Epic, Story, Task

Work comes in sizes. A one-line typo fix and "make every result on screen traceable to its source" are both "work," but pretending they are the same size helps nobody. So we stack work in four layers.

In plain English

Think of a book. **An epic is the book** ("result traceability"). **A story or a bug is a chapter** ("the shear result disagrees between the screen and the report"). **A task is a paragraph** ("add a test that pins the shear number"). And **a release is the shelf** the finished books ship on together ("version 1.0"). You read a book chapter by chapter. You finish an epic issue by issue.



Level	What it is	Lives for	Done when
L1 Release	A version you ship (`v1.0-GA`)	A quarter or two	Every child epic is done
L2 Epic	A large outcome	1–3 months	The outcome is achieved
L3 Story/Bug/Task	One shippable piece	One sprint or less	Merged and verified
L4 Sub-task	A checklist item under an issue	A day or two	Checked off

A word you will hear: a **sub-epic**. When an epic is so big it would hold a hundred child issues, you split it into a few smaller epics. We do this for exactly one epic today (result-traceability), because it touches the largest slice of the backlog. More on that in Part II.

2.3 Backlog

The **backlog** is everything you have agreed is worth doing but have not scheduled yet. It is the "soon, or someday" pile. A healthy backlog is groomed: old items get re-read, big ones get split, dead ones get closed. An ungroomed backlog is just the ninety-three-ticket list again.

2.4 Sprint

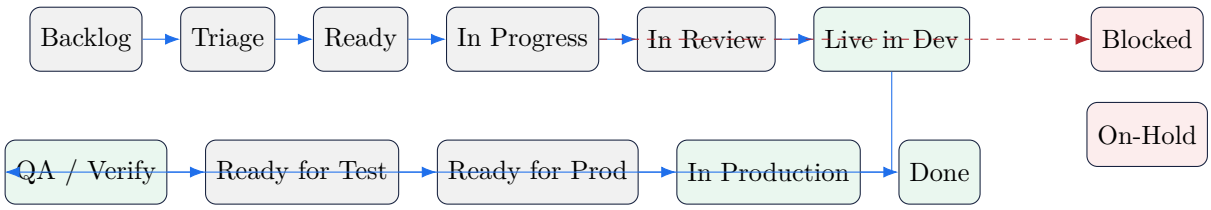
A **sprint** is a fixed, short window of time, two weeks for us, in which the team commits to finishing a chosen handful of issues. The window does not move. If work does not fit, the work moves to the next sprint, not the deadline.

In plain English

A sprint is a fortnightly promise, not a countdown timer on any one task. On Monday you pull a stack of issues you believe you can finish by the following Friday week. Then you protect that stack. The point of the fixed window is rhythm: you always know when the next planning and the next review happen.

2.5 Board, columns, and Status

The **board** is the picture of all your issues as cards, arranged in columns. The column an issue sits in is its **Status**. Our columns run left to right like an assembly line, and *one* line carries a card all the way from "nobody has looked at this" to "it is running in production." There is no second shipped-or-not column: the release stages are simply the tail of the same Status axis.



The first five columns are the everyday development flow. The moment code merges to the default branch it becomes **Live in Dev** — the same card keeps going, now through the release stages: verified (**QA / Verify**), promoted to the test channel (**Ready for Test**), cleared for release (**Ready for Prod**), and finally shipped (**In Production**), before it lands in **Done**. One card, one column, cradle to production.

Blocked and **On-Hold** are off to the side on purpose. They are not stages you pass through, they are places a card waits: *Blocked* when something outside it is in the way (and it must name what it is waiting on), *On-Hold* when the team has deliberately paused it.

2.6 Severity versus Priority

These two get confused constantly, so pin them down now.

- **Severity** is *how bad the problem is.* A wrong structural number shown as if it were right is the worst kind. A misaligned toolbar icon is the mildest.
- **Priority** is *how soon we will deal with it.* It is a scheduling decision, not a fact about the bug.

A tiny cosmetic bug can be high priority the week before a demo. A serious bug can be low priority if it only affects a feature nobody ships yet. Severity is a property of the problem; priority is a choice you make.

2.7 Points and velocity

Some teams put a rough size number on each issue (**story points**: 1, 2, 3, 5, 8) and track how many points they finish per sprint (**velocity**). It is a capacity estimate, not a score. We keep the field available but do not lean on it. Do not treat velocity as a target to maximise; it is a planning aid.

2.8 Milestone and Release

A **milestone** (or release) is a named finish line that groups issues due for the same version, like `v1.0-GA`. "GA" means *general availability*: the version you would hand to a stranger.

2.9 Triage

Triage is the sorting step every new issue passes through: decide its type, its severity if it is a bug, which part of the code it touches, and who owns it. The word comes from hospitals, and the spirit is the same. You are not fixing anything yet. You are deciding what this is and how urgent it is, quickly, so nothing waits in the dark.

CHAPTER 3

Agile and Scrum, in one page

You will hear the words **Agile** and **Scrum**. Here is all you need.

Agile is a way of working that prefers shipping small, useful increments often, and adjusting as you learn, over writing one giant plan up front and following it for a year. **Scrum** is one popular recipe for doing Agile, and sprints come from it.

We borrow the useful parts and skip the ceremony.

We borrow	We skip
Two-week sprints with a planning and a review	Daily stand-up meetings; we are a distributed, part-volunteer team
A groomed backlog and clear ownership	Story-point obsession and velocity targets
Service-level targets measured in *business days*	Any pretence of a 24/7 on-call rotation
A board that shows the true state of the work	Rituals that exist to be performed, not to help

Note

When you read that a serious bug should be triaged "within one business day," that means someone looks at it and sorts it within a working day. It does not mean anyone is paged at 2am. These are promises a volunteer team can actually keep.

Part II

The OsdagBridge System, Concretely

CHAPTER 4

Our four levels, on our project

The generic hierarchy from Part I lands like this on OsdagBridge:

- **Releases** are `v1.0-GA`, `v1.1`, `v2.0`. Most of the current work aims at `v1.0-GA`.
- **Epics** are the ten big outcomes in the next chapter, tagged `type:epic`.
- **Stories, bugs, and tasks** are the everyday issues, each tagged with a `type:`, and for bugs a `sev:`, and one or more `area:` labels.
- **Sub-tasks** are checklist items linked under an issue as native GitHub sub-issues.

CHAPTER 5

Labels: three decisions per issue

A **label** is a coloured tag on an issue. We use three families of them, and the rule is simple: a well-triaged issue answers three questions through its labels. *What kind of thing is it? If it is a bug, how bad? Which part of the code?*

5.1 `type:` what kind of work

Label	Meaning
`type:epic`	An outcome-level container. Closes when the outcome is achieved.
`type:feature`	New capability a user asked for.
`type:story`	A sprint-sized piece of user value.
`type:bug`	Something behaves incorrectly.
`type:task`	Sprint-sized work with no direct user story.
`type:chore`	Housekeeping (dependencies, cleanup).
`type:spike`	A time-boxed investigation to answer a question.
`type:docs`	Documentation.
`type:test`	A test to be written, often filed by a bot when a bug closes.

5.2 `sev:` how bad the bug is

Bugs, and only bugs, also get a severity. The four tiers get their own chapter below because the definitions carry real weight.

5.3 `area:` which part of the code

`area:` is the subsystem the issue touches, and it is *multi-valued*: an issue routinely spans two areas, and that is allowed. Examples you will see: `design-core`, `analysis-opensees`, `reports`, `ifc-boq`, `packaging`, `memory-stability`, `ci-cd`, `docs`, plus the user-interface areas carried over from the old tracker.

Note

You may notice the board has fields for **Priority**, **Size**, **Epic**, and **Target release** too, but there are no `pri:` or `size:` *labels*. That is deliberate. Those four live as board fields for now (Part III), and only graduate to labels once the system can fill them in reliably. Shipping eight label families onto a young backlog would overload the one thing that already works.

CHAPTER 6

The ten epics

Every issue should belong to an epic, so the board can roll individual tickets up into "are we actually moving the big outcomes." Here they are.

	Epic	Outcome (done when…)	Release
E1	result-traceability	One value, one source; a disagreement fails a test	v1.0-GA
E2	input-integrity	Inputs are traceable, persist, and reset to defaults cleanly	v1.0-GA
E3	windows-stability	No crashes or memory blowups across repeated design runs	v1.0-GA
E4	verification-harness	Hand-checked engineering cases run as tests; core is covered	v1.0-GA
E5	ci-quality-gates	Every pull request is gated by tests and lint	v1.0-GA
E6	release-ga	A reproducible installer on Windows and Linux	v1.0-GA
E7	reporting-boq	Report chapters, bill of quantities, and IFC all agree	v1.0-GA
E8	cad-visualization	3D model, cross-section, and plots stay in sync	v1.0-GA
E9	docs-onboarding	Accurate README, real guides, contributor docs	v1.1
E10	architecture-debt	Break up oversized modules; make the solver layer real or remove it	v1.1

E1 is split into six sub-epics, one per design-check family, because it maps to the largest part of the backlog and will grow past GitHub's limit of one hundred child issues if left whole:

flexure	**shear**	**LTB**
fatigue	**stiffeners**	**shear-connectors**

Each is its own epic issue, linked under E1 as a native sub-issue, so E1's progress bar is really the sum of six smaller bars.

CHAPTER 7

Bug tiers and what we promise

Severity is where a structural-engineering project earns its keep. The defining rule:

Open-source gotcha

S1 is reserved for wrong numbers shown as if they were right. A crash is loud; you know it failed. A wrong capacity presented as correct is silent, and in bridge software it is the one that can hurt someone. That is why it sits at the top.

Severity	Means	Look within	Fix within
`S1-critical`	Wrong structural output shown as correct; a crash; a release blocker	1 business day	Current sprint
`S2-major`	A feature is unusable, or a value cannot be verified; a workaround exists	2 business days	Current sprint
`S3-minor`	Incorrect display, no safety impact	5 business days	Within 2 sprints
`S4-cosmetic`	Polish: hover states, alignment, wording	Next grooming	When convenient

"Look within" is the triage promise: how fast someone sorts it. "Fix within" is the target for shipping the fix. Both are business days, and both are promises a part-volunteer team can keep. They are not a pager.

Part III

Setting Up the Board, Step by Step

This part builds the board from nothing. If your board already exists, you can read this as a tour of why each piece is there. If you are standing up a fresh copy, do it in order.

CHAPTER 8

Before you click anything

You need three things.

1. **A GitHub account** that owns, or can administer, the repository.
2. **The repository itself**, `Osdag-Project-Management`. It is deliberately its own repository and not a fork, because GitHub refuses to run scheduled jobs in forks, and the nightly reconciler is a scheduled job.
3. **A fine-grained access token** for the automation. This is a password-like string GitHub generates. It grants the robots permission to edit the board.

Open-source gotcha

Never paste the token into chat, a commit, an issue, or this handbook. It is stored once, as a repository secret named `GH_PM_TOKEN`, and referenced by name. A token that leaks into a public repository is a token you must revoke and regenerate the same hour. Treat it like the key to the building.

One more fact that shapes the whole setup: GitHub's newer "Projects" boards can be created and configured two ways, by clicking, and through GitHub's programming interface. A few field types (the sprint timeline and the two date fields) and all of the saved views can *only* be made through the programming interface at the tool versions we target. That is why our setup uses a small script for those, and your own clicks for the rest. Both are described below.

CHAPTER 9

Create the project

1. On GitHub, go to your account's **Projects** tab and click **New project**.
2. Choose the **Table** template (you will add views later) and name it after the project — **OsdagBridge** (the board title is the overlay's `display_name`).
3. Note the number it is given (ours is #3) and its address. You will point the automation at this.

In plain English

At this moment you have an empty spreadsheet that happens to live on GitHub. The next two chapters give it columns (fields) and saved filters (views). After that it becomes a board.

CHAPTER 10

Add the board fields

A **field** is a column: one piece of structured information every card can carry. We use thirteen. For each one below you get what it is, why it exists, its type, and its values.

Note

Three of these (**Sprint**, **Start date**, **Target date**) can only be created through the automation, not the click menu, at our tool versions. The bootstrap script (`python -m project_management.bootstrap_board -software osdagbridge`) builds them all from `config/base.yml` (the shared schema) merged with the project's `config/software/osdagbridge.yml` (its areas and epics) in one shot, and running it twice changes nothing the second time. If you are doing it by hand, create the single-select and number fields yourself and let the script add the three special ones.

Field	Type	Why it exists / values
Status	single-select	Which column the card is in — the whole life of a card on one axis. Backlog, Triage, Ready, In Progress, In Review, Live in Dev, QA / Verify, Ready for Test, Ready for Prod, In Production, Blocked, On-Hold, Done.
Sprint	iteration (14 days)	Which fortnight this is committed to. A timeline field; automation-only.
Type	single-select	What *kind* of work it is: Epic, Feature, Story, Bug, Task, Chore, Spike, Docs, Test. Mirrors the `type:` labels so the board can *group* by kind.
Priority	single-select	How soon: P0-now, P1-sprint, P2-next, P3-backlog.
Severity	single-select	How bad: S1 to S4 (bugs).
Size	single-select	Rough effort: XS (≤2h), S (≤1d), M (≤3d), L (≤1wk), XL (split it).
Points	number	Optional numeric size: 1, 2, 3, 5, 8.
Epic	single-select	Which big outcome (E1 to E10).
Area	**multi-select**	Which subsystems. Multi-valued on purpose: issues span two.
Team	single-select	Which team owns it (UI, Design Core, ...). Derived from the project's `teams:` block.
Target release	single-select	v1.0-GA, v1.1, v2.0, future.
Start date	date	When work began. Feeds the roadmap view.
Target date	date	When it is due. Feeds the roadmap view.

In plain English

One axis, not two. Earlier versions of this board split "is it written?" from "is it shipped?" into two separate columns — a Status field and a Deploy stage field. We folded them into one. Now a single **Status** carries a card from Backlog all the way to In Production: once code merges it becomes *Live in Dev*, and the release stages (*Ready for Test*, *Ready for Prod*, *In Production*) are just the far end of the same line. The win is that there is only ever one honest answer to "where is this?" — you never have to reconcile two columns that disagree. (If you are looking at an older board that still has a separate Deploy stage field, the reconciler leaves it in place — it never deletes — but nothing new is written to it.)

Open-source gotcha

Area is multi-select, not single-select, and that choice matters. Most real issues touch two subsystems at once. A single-select Area field forces a lie (pick one) and quietly breaks any "show me everything touching reports" view. If you rebuild this field by hand, make it multi-select.

Open-source gotcha

Without the two date fields, the roadmap view renders empty. The roadmap layout draws bars on a timeline, and a bar needs a start and an end. Leave these out and the Epic Roadmap view looks broken for no obvious reason. This is a classic half-hour of confusion; the fix is simply to add both date fields.

CHAPTER 11

Build the nine views

A **view** is a saved arrangement: a layout plus a filter, so a specific audience sees exactly the cards they care about. All nine are created by the automation, because the click menu cannot save a filter with a new view at our tool versions. Here is each one and, just as important, when it is *supposed* to look empty.

View	Layout	Shows	Expected empty when...
Current Sprint	Table	This fortnight's committed, unfinished work, as a checklist	No issues assigned to a sprint yet
Workload	Board	This sprint's open work grouped by person (this absorbs the old "By Owner")	No sprint work yet
By Team	Board	Every open issue grouped by which team owns it	No open issues
By Type	Board	Every open issue grouped by kind (epic, bug, feature, ...)	No open issues
Triage Queue	Table	New issues needing sorting, oldest first	Everything has been triaged
Backlog Grooming	Table	Open, non-epic issues with no sprint	Nothing is unscheduled
Epic Roadmap	Roadmap	The epics on a timeline	Never, once date fields exist
Release v1.0-GA	Board	Everything aimed at the next release	
Release pipeline	Board	Items in the release tail of Status, moving through the conda channels	Nothing has merged to dev yet

Note

An empty Current Sprint is not a broken board. Until you have actually planned a sprint and dragged issues into it, that view is supposed to be empty. People see it on day one and assume something failed. Nothing failed. Part IV fills it.

Open-source gotcha

The grouped views need one manual toggle each. The automation sets each view's *filter*, but GitHub exposes no way for a program to set a view's *grouping*. So after setup,

open each of these, click **... -> Group by**, and pick its field: **Workload -> Assignees** (or a custom Owner field — see the interns chapter in Part V); **By Team -> Team**; **By Type -> Type**; and **Release pipeline -> Status**. Same class of one-time click as the five workflows in the next chapter.

CHAPTER 12

Turn on the five automations you click yourself

Here is a limit worth understanding rather than fighting. GitHub's built-in board automations (the little "when this happens, do that" rules) cannot be created through the programming interface. GitHub only lets a program *delete* them or *read* them, never create or edit. So these five are the one part of setup that is genuinely manual, once per board, by hand.

Open the board, click the `...` menu in the top right, and choose **Workflows**. Each rule below is off by default. Turn on all five.

1. **Auto-add items from the repository.**
   Workflows -> **Auto-add to project** -> Edit. When an issue is added to `Osdag-Project-Management` (filter `is:issue`), add it to the project. Enable.
   Why: without this, new issues never reach the board at all.

2. **Item closed becomes Status: Done.**
   Workflows -> **Item closed** -> Edit. Set Status to Done. Enable.

3. **Pull request merged becomes Status: Live in Dev.**
   Workflows -> **Pull request merged** -> Edit. Set Status to Live in Dev. Enable.
   Why: on the single-axis board a merged PR is not finished, it is *shipped to the dev channel*. The release pipeline then carries it on through the tail (Ready for Test -> In Production); closing the issue is what finally moves it to Done.

4. **Auto-archive Done items after 14 days.**
   Workflows -> **Auto-archive items** -> Edit. When Status is Done and untouched for 14 days, archive it. Enable.
   Why: keeps the board about live work, without deleting history.

5. **Item added becomes Status: Triage.**
   Workflows -> **Item added to project** -> Edit. Set Status to Triage. Enable.
   Why: every arrival starts in the triage queue, so nothing skips sorting.

Check it worked

Open a throwaway test issue in the repository. Within a few seconds it should appear on the board in the **Triage** column. Close it; it should move to **Done**. If both happen, all five rules are live. Delete the test issue when you are done.

Open-source gotcha

Do not set field automations here. The click menu can also auto-set fields, but our field logic is driven from configuration files and checked nightly. Setting the same thing in two places is how a board starts disagreeing with itself. These five workflow rules are the *only* thing you turn on by hand.

CHAPTER 13

Is my board alive?

Once set up, the board mostly runs itself, watched by a nightly job (the **reconciler**, Part V covers what it is and why). You do not need to babysit it, but you should know the one health signal.

Every night the job writes a timestamp into a file called `.heartbeat` and commits it. If the job ever silently stops running, that timestamp stops advancing, and the next run that does fire notices it is stale and raises a "Board health" issue. So the way you check the board is alive is simple: the heartbeat file's date should be recent, and there should be no open "Board health" issue complaining that it is old.

In plain English

It is a dead-man's switch. A guard who has to press a button every hour; if the button stops being pressed, an alarm rings. The heartbeat is that button, and it is committed into the repository so a stopped robot leaves a visible trail instead of just going quiet.

Part IV

Running Sprints and Daily Use

CHAPTER 14

How to use it: your first day on the board

The board is built, the fields are in place, the views exist. Now: what do you actually *do* with it? This chapter is the hands-on part. If you have read nothing else, read this. Everything after it goes deeper on the same actions.

Everything below happens in the browser, on the project board. You do not need the command line, a token, or any of the configuration files to do daily work — those are for the person who *sets up* the board, not the person who *uses* it.

14.1 Open the board and get your bearings

Open the project. Along the top you will see tabs — these are the **views** from the last part. You will live in a few of them:

Tab	Open it when you want to...
Current Sprint	See this fortnight's committed work as a checklist — your day-to-day home.
Workload	See who is carrying what, grouped by person. Find your own name.
By Team	See everything your team owns, in one place.
Triage Queue	Sort brand-new issues nobody has classified yet.
By Type	Scan all the bugs, or all the epics, at once.

In plain English

A view never hides or changes an issue — it is just a lens. The same card can show up in Current Sprint, Workload, and By Team at once; they are three windows onto one pile, not three piles. Switching tabs never moves anything.

14.2 Find the work that is yours

Open **Workload** and look for your name — your issues are grouped under it. Nothing there? Then you have not been given a sprint issue yet; ask at planning, or pull one from **Current Sprint** that has no owner. If you do not have a name column at all, you have not been added to the board yet — see the interns chapter in Part V for the one-time invite.

14.3 Pick up an issue and move it across

This is the single most common thing you will do. An issue moves left-to-right along **Status** as you work it:

1. **Take it.** Open the card, set yourself as owner (the **Assignees** field, or the custom Owner field — see Part V), and drag it from **Ready** to **In Progress**.

2. **Work it.** Write the code, open a pull request. When the PR is up for review, move the card to **In Review**.

3. **Merge it.** When the PR merges, the board moves the card to **Live in Dev** for you (that is one of the five automations). You do not drag it yourself.

4. **Let it ride the tail.** From there the release pipeline carries it on through **QA / Verify**, **Ready for Test**, **Ready for Prod**, and **In Production** as it is promoted through the conda channels. Closing the issue is what finally sends it to **Done**.

5. **Stuck?** If something outside your control is in the way, move the card to **Blocked** and write *what* you are waiting on in a comment. If the team has deliberately parked it, use **On-Hold**.

Open-source gotcha

Do not skip a card straight to Done because "the code is written." On this board Status is one honest line from idea to production. "Written and merged" is **Live in Dev**, not Done. Done means it has been through the release tail and closed. If you jam things to Done early, the Release pipeline view goes blind and nobody can tell what has actually shipped.

14.4 File a new issue

Found a bug, or thought of work that needs doing? Open an issue in the source repository (not on the board — the board pulls it in automatically). Give it a clear title and a body that says, for a bug, *what you did*, *what you expected*, and *what happened instead*. Within seconds it appears on the board in the **Triage** column, waiting to be classified. That is the automation doing its job; you do not add it to the board by hand.

14.5 Fill in the fields (classify a card)

When you triage or pick up a card, set its fields. This is what turns a flat list back into a managed project. The ones you touch most:

Field	What to set
Type	What kind of work it is: Bug, Feature, Story, Task, ...
Severity	For bugs only: S1 (wrong answer shown as right, or a crash) down to S4 (cosmetic).
Area	Which subsystem(s) it touches — pick more than one if it spans two.
Team	Which team owns it.
Epic	Which big outcome it belongs to.
Size	Rough effort. Anything **XL** should be split, not started.
Priority	How soon: P0 now, down to P3 backlog.

In plain English

You do not have to fill in every field the moment an issue is born. Triage sets the few that matter (Type, Severity, Area, an owner); the rest can wait until the issue is actually pulled into a sprint. An empty **Points** or **Target date** is not a mistake — it just means "not decided yet."

14.6 A worked first hour

Say you are handed issue #312, "girder self-weight looks wrong in the report."

1. It is sitting in **Triage**. You open it and classify: **Type** = Bug, **Severity** = S1 (a wrong structural number shown as if correct), **Area** = `reports` and `design-core`, **Team** = Reports & BOQ, **Epic** = E1 result-traceability. You set yourself as owner.
2. At planning it is pulled into the sprint and you drag it to **In Progress**.
3. You find the bad key binding, fix it, and open a PR. You move the card to **In Review**.
4. The PR merges; the board moves #312 to **Live in Dev** on its own.
5. It rides the release tail to **In Production**. You close the issue and it lands in **Done** — fixed, shipped, and traceable to the clause it implements.

Check it worked

You have used the board correctly if, at any moment, someone can open it and answer three questions without asking you: *what are you working on* (your card in In Progress), *is it stuck* (Blocked, with a named reason), and *has it shipped* (its place in the release tail). If all three are answerable from the board alone, you are doing it right.

CHAPTER 15

The fortnightly rhythm

Sprints are two weeks, starting on a Monday. The rhythm repeats, and its whole value is that it repeats, so nobody wonders when the next planning or review is.

When	What happens
Sprint start (Mon)	**Planning**: pull a realistic stack of issues into the sprint and give each an owner.
Mid-sprint	**Grooming**: re-read the backlog, split anything too big, close anything dead.
Sprint end (Fri)	**Review and retro**: check what shipped, verify it, and note one thing to do better.

15.1 Planning: the pull

Do not push work onto people. Let the sprint *pull* from the top of the prioritised, ready backlog until it is reasonably full. An issue is eligible to be pulled only if it is **Ready** (see the Definition of Ready below). Give every pulled issue exactly one owner.

15.2 Grooming: split the XLs

Any issue sized **XL** is a sign it is really several issues wearing a trench coat. Split it. A groomed backlog is one where the top is always ready to pull and the bottom is honestly closed if it is never going to happen.

Note

Priority is not the sprint. An issue can be high priority and still not be in this sprint, because priority says "deal with this soon" and the sprint says "these specific things, this fortnight." Do not conflate the two fields.

CHAPTER 16

Triage: three decisions and an owner

When a new issue lands in the Triage column, you make three quick decisions and pick an owner. You are not solving it. You are classifying it so it stops being invisible.

1. **Type.** Bug, feature, task, chore, docs? Add the `type:` label.
2. **Severity** (bugs only). S1 to S4, using the definitions from Part II. Remember the S1 rule.
3. **Area.** Which subsystem(s)? Add one or more `area:` labels.
4. **Owner and epic.** Who holds it, and which epic it rolls up to.

Open-source gotcha

An unreproduced bug is not Ready. If you cannot make the bug happen, the issue's job is to capture how to reproduce it, not to be scheduled for a fix. "I can't reproduce this" is a valid, common triage outcome, and it is more useful than a hopeful guess.

Note

Issues opened through the automation or the programming interface skip the nice input form a human would fill in. So triage cannot assume the form's fields are present. Read the issue, do not trust a template to have run.

CHAPTER 17

The life of one issue

Here is the whole journey, using a real one: issue `#294`, "the shear capacity shown in the app disagrees with the shear capacity in the generated report." This is a textbook S1: two numbers that should match, do not, and the software presents both as correct.

1. **Filed.** Someone reports it. It lands on the board in **Triage** automatically.
2. **Triaged.** You label it `type:bug`, `sev:S1-critical`, `area:design-core` and `area:reports` (it spans both), assign an owner, and roll it up to the shear sub-epic of E1. Because it is S1, the promise is: sorted within a business day, fixed this sprint.
3. **Made Ready.** The owner reproduces it and writes down the exact steps and the two disagreeing numbers. Now it meets the Definition of Ready and can be pulled.
4. **Planned.** At sprint planning it is pulled into the current sprint. Status moves to **Ready**, then **In Progress** when work starts.
5. **In Review.** The fix opens a pull request; Status becomes **In Review**. A merged pull request will move the card to Done automatically, but we are not done yet.
6. **QA / Verify.** After merge, someone confirms the app and the report now show the same shear number, and that a *test* now guards it, so the bug cannot silently return.
7. **Done.** Verified and closed.

17.1 Definition of Ready

An issue is Ready to be pulled into a sprint when:

- It has a type, and if a bug, a severity and reproduction steps.
- It has an area and an owner.
- It is small enough to plausibly finish in one sprint (not XL).
- Anyone could pick it up and know what "done" means.

17.2 Definition of Done

An issue is Done when:

- The change is merged.
- For a bug, a regression test now guards the fix.
- For a wrong-number bug, a verification case pins the correct value by hand.
- It has been checked against the original report, not just assumed.

In plain English

The two definitions are gates, not paperwork. “Ready” stops half-formed issues from clogging a sprint. “Done” stops “it works on my machine” from counting as finished. For structural software, the Done rule about a guarding test is the difference between fixing a wrong number once and fixing it every time it comes back.

Part V

Running It in the Open

OsdagBridge is open source, and the board is public. That raises a handful of fair questions — can outsiders tamper with it, how do interns join without repository access, who is allowed to change or delete work, how do you see who is overloaded, and were all those robots really necessary. Here are straight answers.

CHAPTER 18

Can outsiders tamper with our board?

Short answer: no. They can watch, they cannot touch.

A GitHub project has a **Visibility** setting, in Settings under a section literally labelled "Danger zone," set to either **Public** or **Private**.

- **Public** means anyone on the internet, even signed out, can *view* the board. Read only.
- **Private** means only people you have granted at least read access can see it at all.

Viewing is not editing. Only people you have explicitly added to the project with **Write** or **Admin** access, plus the owner, can move a card or change a field. A random passer-by has no edit button. Even changing the visibility setting itself is restricted to project admins.

In plain English

Public is a shop window, not an open till. People walking past can see everything laid out inside. The door to rearrange the display is locked, and only staff have a key. Making the window public does not unlock the door.

So yes, if the board is public, the whole world can *watch* your progress. For an open-source project that is usually a feature: contributors and users can see what is being worked on. If you would rather not be watched, set it to Private and only invited people see anything. Either way, outsiders cannot change it.

Note

There is a second safety net. Even a trusted collaborator can make an honest mistake, drag the wrong card, delete a field option. The nightly reconciler compares the live board against the configuration files and, if they disagree, opens a "Board drift detected" issue describing exactly what changed. It never silently edits anything back. So an accidental change is caught and surfaced, not hidden.

CHAPTER 19

How do 20 to 25 interns use the board without repository access?

This is the real operational question, and the good news is that **project access is separate from repository access**. GitHub's own words: adding someone to a project "only affects collaborators for your project, not for repositories." So you can let interns run the board without making any of them repository collaborators.

19.1 Getting them onto the board

On the board, open the **...** menu -> Settings -> **Manage access** -> **Invite collaborators**. Add each intern with the **Write** role, which GitHub defines as "view and edit the project." That is enough to move cards and set Status, Sprint, and any custom field. No repository collaborator status needed.

Because our repository is **public**, the interns can also *see* every issue on the board. (Viewing an individual card follows the card's repository permissions; on a public repository, that means everyone.)

19.2 The one friction: getting assigned

There is a single catch, and it is about GitHub's built-in **Assignees** field, the little avatars on an issue. GitHub only lets you assign an issue to: yourself, anyone who has *commented* on it, anyone with write access to the repository, or an organisation member with read access. An intern who is none of those cannot be a native assignee yet. You have three ways around it, and this handbook does not pick one for you.

Option	How it works	Best when...
A. Custom "Owner" field	Add a project field (a single-select of names, or a text field) and use it instead of native Assignees. Anyone with project Write can set it. Zero repository access.	You want the simplest thing that works today.
B. "Comment once"	Each intern comments once on the repository, which makes them eligible for the native Assignees field from then on.	You want to keep GitHub's built-in avatars.

Option	How it works	Best when...
C. Move to an organisation	Put the board under a GitHub *organisation* (such as `osdag-admin`). Organisations support *teams* and *base roles*: one "interns" team gets read on the repo (so they are assignable) and write on the project.	You are ready to promote the project and want it to scale cleanly to 25 people.

Open-source gotcha

A personal account cannot invite whole teams to a project, only individuals, one at a time. Inviting twenty-five interns by hand is tedious and easy to get wrong. Organisations remove that friction: you manage one team, not twenty-five invitations. That is the strongest practical argument for Option C, and it lines up with the eventual move of the whole project to `osdag-admin`.

In plain English

For a term or two on the current setup, Option A (a custom Owner field) is the least fuss: it needs nothing but project access and works the same for one intern or fifty. Option C is where you want to end up once the project graduates to an organisation. Option B is a quick bridge if you specifically want the built-in avatars in the meantime.

CHAPTER 20

Who can change what: leads, contributors, and deletion

The model the team asked for is: **everyone can see the board, everyone can file an issue, only leads can drive the board, and nobody can quietly delete someone's work.** GitHub gives you exactly this, because it keeps three separate permissions apart. The trick is knowing which lever does which.

Who wants to...	**Can they, by default?**	**Where it is controlled**
See the board	Yes, anyone, if the project is **Public** (read-only)	Board Settings -> Danger zone -> Visibility
Move cards, set fields	Only project collaborators with **Write/Admin**	Board Settings -> Manage access
Open an issue	**Anyone** with a GitHub account, on a public repo	Repo settings (issues can be disabled — rarely wanted)
Close / reopen / edit an issue	Repo **Write** (and the author, on their own)	Repo -> Settings -> roles
Delete an issue	Only repo **Admins**	Repo -> Settings -> roles

20.1 Give the leads the board, not the repo

This is the key move. *Project access is separate from repository access*, so a team lead can have full control of the sprint board **without any write access to the code.** On the board: ... -> Settings -> **Manage access** -> **Invite collaborators** -> add each lead with the **Write** role. They can now move cards and set Status/Sprint/Priority/Owner. Reserve **Admin** for whoever also manages the project's settings and the board workflows.

20.2 Everyone else needs no setup

Because the source repositories are public, anyone can already *open* an issue — it lands on the board through the Auto-add workflow. Contributors do **not** need any board access to file work; they only need it to *edit the board*, which is precisely what you are withholding. So "anyone can add, only leads can arrange" is the default state, not something you configure.

Open-source gotcha

"Add" and "delete" are deliberately different powers. Filing an issue is open to the world; *deleting* one needs repo **Admin**. Repo Write can close, reopen, and edit but cannot

delete. So keep the Admin role to the two or three people who actually own each repository, and "everyone can add, nobody can delete" holds by itself. The nightly reconciler is the backstop: if even a trusted lead makes a mistaken change, it surfaces in the "Board drift detected" issue rather than passing unnoticed.

In plain English

Picture a public noticeboard in a town hall. Anyone can walk in and pin up a notice (open an issue). Only the staff on rota can rearrange the board into sections (leads with project Write). And only the hall manager can take a notice down for good (repo Admin deletes). Three different keys, three different people — which is exactly why no single role can both run the board and quietly erase history.

CHAPTER 21

Who is carrying the most: the Workload view

At sprint review you want one glance that answers "who is overloaded and who has room?" That is the **Workload** view. It ships with the board (filtered to the current sprint's open items); you turn its grouping on once, by hand, because GitHub has no programming interface for a view's grouping.

1. Open the **Workload** tab.
2. **... -> Group by -> Assignees** (or **Owner**, if you use the custom Owner field from the interns chapter).

Each person becomes a column and the column's height is their open load this sprint. The tallest column is the most-loaded person; an empty column is someone with capacity. Use it in the review to rebalance before the next planning pull.

Note

For a *trend* rather than a snapshot, the board's **Insights** tab (top-right) charts item counts by assignee over time — also a UI-only feature. And if your team cannot use GitHub's native Assignees (interns without repo access, from the previous chapter), group Workload by the custom **Owner** field instead; it reads the same.

CHAPTER 22

Were all those automations necessary?

Fair question. The honest answer: **one of them is load-bearing, the rest earn their place for different reasons, and you could run a smaller board without most of them.** Here is the accounting.

Automation	What it buys	Verdict
The reconciler (`pm-reconcile`)	Keeps labels and board matching the config, detects drift, checks liveness	**Load-bearing.** This is the one that makes the board trustworthy over time.
Board bootstrap (`pm-bootstrap-board`)	Builds the whole board from config in one command	One-time, but valuable: the board is reproducible, not clicked from memory.
Epic creation (`pm-epics`)	Creates the epic issues and sub-epics from config	One-time; same reproducibility argument.
Board populate (`pm-bootstrap-board -populate`)	Adds the source repo's open issues onto the board, in place	One-time backfill; replaces the retired "seeder" that used to copy issues into the PM repo.
The five manual workflows	Auto-add, close-to-Done, archive, triage-on-arrival	**Necessary.** These are what let the board maintain itself.
Health / heartbeat	Notices if the reconciler silently dies	Insurance. Cheap, and it matters most when nobody is watching.

In plain English

Strip it to the bone and a working board needs three things: the board itself, the five manual workflow rules, and the labels reconciler. Everything else buys one of three things: reproducibility (so the board can be rebuilt or promoted from config instead of somebody's memory), a one-time migration, or safety when the system runs unattended. None of it is decoration, but not all of it is essential on day one. For a team of twenty-five people who

look at the board daily, the health robot is the most skippable; for a board left alone over a quiet holiday, it is the one you are glad you built.

The pieces that are *not yet built*, the sprint report, the epic roll-up robot, and the analytics bots, were deferred on purpose. They produce derived summaries, and there is no point generating a sprint report before any sprint has run.

Part VI

Reference

CHAPTER 23

Glossary

Agile
A way of working that ships small useful increments often and adapts, rather than following one big up-front plan.

Area
A label for which subsystem an issue touches. Multi-valued.

Backlog
Agreed work that is not yet scheduled.

Board
The visual arrangement of issues as cards in columns.

Definition of Done
The checklist an issue must meet to count as finished.

Definition of Ready
The checklist an issue must meet before it can be pulled into a sprint.

Drift
The board no longer matching its configuration files.

Epic
A large outcome that spans months and contains many issues.

Field
A structured column on the board (Status, Sprint, Severity, and so on).

GA
General availability: the version you would give a stranger.

Grooming
Regularly re-reading and tidying the backlog.

Heartbeat
A committed timestamp proving the nightly job is still running.

Issue
One unit of work or one conversation.

Milestone
A named release that groups issues due for the same version.

Priority
How soon we will deal with something. A scheduling choice.

Reconciler
The nightly job that keeps the board matching the config and reports drift.

Scrum
A popular recipe for Agile; sprints come from it.

Severity
How bad a bug is. A property of the problem.

Sprint
A fixed two-week window of committed work.

Story
A sprint-sized piece of user value.

Sub-epic
A smaller epic split out of a very large one.

Triage
Sorting a new issue: type, severity, area, owner.

Velocity
How much work a team finishes per sprint. A capacity estimate, not a target.

View
A saved layout-plus-filter for a specific audience.

CHAPTER 24

Which file controls what

The board is driven by configuration, not clicks. When you want to change the *structure* of the system, you edit a file and let the reconciler apply it, rather than editing the board directly. Here is the map.

Configuration lives in two layers that the loader stitches together: one shared base, and one small file per project. That split is what lets a second FOSSEE project get its own board without copying the whole schema (see the next chapter).

File	Controls
`config/base.yml`	**Shared across every project**: the `type:` and `sev:` labels, all the board fields (Status, Sprint, Type, Priority, . . . , Team), the sprint schedule, and the nine views.
`config/software/osdagbridge.yml`	**This project only**: its display name, its `area:` labels, its epics, its `teams:`, its `source_repos`, and its conda channels. The board's Epic, Area, and Team options are *derived* from here, so you never write them twice.
`config/rollup.yml`	The "All Projects" roll-up board and its `Software` field options.
`docs/pm/SOP-01..10`	The terse checklists this handbook expands on — including SOP-09 (customising) and SOP-10 (standing up on osdag-admin).
`docs/pm/BOARD-SETUP.md`	The five manual workflow clicks, as a checklist.
`docs/pm/PROMOTION.md`	The plan for moving the board to `osdag-admin`. Still needs named human owners.

Note

The old single-file layout is gone. Earlier versions kept `config/labels.yml`, `epics.yml`, `project.yml` and a `seed.yml`. Those were merged into `base.yml` + the per-project overlay, and the copy-issues-in "seeder" was retired in favour of tracking each repo's issues *in place* (see the intake SOP). If you find a reference to those old files anywhere, it predates this handbook.

Note

Changing configuration is reviewed like code. A change to the taxonomy goes through a pull request, so a second person sees it before it touches the live board. The reconciler never deletes anything on its own; if the board has something the config does not, it *reports* it and waits for a human, because some deletions (a label coming off every issue that used it) cannot be undone.

CHAPTER 25

Changing the board: the manual way and the YAML way

Almost everything about the board can be changed **two ways**, and they coexist safely:

- **The YAML way** — edit a file in `config/`, open a pull request, run the workflow. Reviewable, repeatable, and it survives a rebuild from scratch. This is how the board is promoted to another repo unchanged.
- **The manual way** — click it in GitHub. Instant, no pull request, but invisible to the reconciler and lost if the board is ever rebuilt from config.

In plain English

The golden rule is what makes the two safe together: **the reconciler never deletes.** Anything you add by hand that the config does not know about is *reported* as an extra, never removed. Anything the config declares that is missing gets added. So you can hand-tweak freely and write it into YAML later, or never — nothing you clicked gets clobbered except the few fields config explicitly owns (Status options, the sprint schedule, each select field's options, and label colours).

Change	YAML way	Manual way
A label or area	`base.yml` (type/sev) or the overlay (area), then `pm-reconcile`	Issues -> Labels -> New (reported as an extra until you add it to config)
An epic	overlay `epics:`, then `pm-epics`	Open an issue with `<!- pm-epic:KEY ->` in the body; the reconciler adopts it
A board field or option	`base.yml fields:`, then `pm-bootstrap-board`	Board -> + -> New field
Sprint length or start	`base.yml` Sprint field, or `-sprint-start`	Board -> Sprint field -> Edit iterations
A whole new project	new `config/software/<name>.yml`	— (config is the only sane path)

The full worked examples live in `SOP-09`. Two of them matter enough to spell out here.

25.1 Changing the sprint length (7, 14, or 15 days)

Sprints are two weeks by default. The length is a single number — `duration_days` — on the Sprint field in `config/base.yml`:

```
- name: Sprint
  type: ITERATION
  duration_days: 14        # 7 = weekly, 15 = a fortnight-plus
  start_date: "2026-08-03"
```

1. Change `duration_days` — `7` for weekly sprints, `15` for slightly longer ones.
2. Open a pull request, merge it.
3. Run **Bootstrap board** (`pm-bootstrap-board`). GitHub regenerates the numbered iterations *forward* from `start_date`, so this reshapes future sprints.

Note

Where the numbering *starts* is separate from how *long* each sprint is. To move where sprint 1 begins — for a fresh demo, "start today" — do *not* edit `start_date` by hand. Run bootstrap with `-sprint-start today` (or a specific `-sprint-start YYYY-MM-DD`). It sets only the first iteration's start; the length is untouched. That is exactly what a first setup on `osdag-admin` uses, so the demo's first sprint begins on the day you stand it up (see `SOP-10`).

The manual alternative is Board `->` **Sprint** field `->` **Edit iterations**, changing the duration or adding iterations by hand. It is instant, but the config's `duration_days` still wins the next time anyone runs Bootstrap board — so if you want a length change to stick, change both, or just change the YAML.

25.2 Adding a whole new project

This is the payoff of the base-plus-overlay split: a second board — **Osdag**, **Osdag-web**, any FOSSEE project — is essentially one new file.

1. Copy `config/software/osdagbridge.yml` to `config/software/<name>.yml`.
2. Edit its `display_name`, `source_repos`, `area_labels`, `epics`, and `conda_channels`. Leave `board_number` blank until the board exists.
3. Add `<name>` to `config/rollup.yml` so it appears on the "All Projects" roll-up.
4. Open a pull request, merge it.
5. Run **Bootstrap board** with `-software <name>` — it creates the board, links every source repo, and builds the fields and views. Add `-sprint-start today` for a demo.
6. Run **Epics** and **Reconcile** with the same `-software <name>`.
7. Do the manual clicks from `BOARD-SETUP.md`: one **Auto-add** per source repo, plus the workflow rules.

In plain English

Nothing about the engine changes to add a project — same commands, a different `-software` name. The shared `base.yml` supplies the labels, fields, sprint schedule, and views; the overlay supplies only what is genuinely this project's own. That is what "one engine, many boards" means in practice.

CHAPTER 26

Where to go next

- New contributor? Read Parts I and II, then file or pick up one issue and walk it through the lifecycle in Part IV.
- Standing up a board? Follow Part III in order, then confirm with the checks in each "Check it worked" box.
- Running triage or planning? Parts IV, and the matching `SOP-01` and `SOP-03` for the terse version.
- Customising the board — a new label, a new epic, a different sprint length, or a whole new project? The chapter above, and `SOP-09` for the full manual-vs-YAML worked examples.
- Standing the board up on `osdag-admin`, with access control for leads and interns? `SOP-10` is the click-by-click runbook.
- Thinking about promotion to an organisation? Read `PROMOTION.md`. It is the one genuinely unfinished piece, and it needs people, not code: a named contact, an owner for the canonical tracker, and a decision about the existing issue history.

Cheat sheet

The whole system on one page

Status columns (one axis): Backlog -> Triage -> Ready -> In Progress -> In Review -> Live in Dev -> QA / Verify -> Ready for Test -> Ready for Prod -> In Production -> Done. *Blocked* and *On-Hold* wait to the side; a blocked card names its blocker. The release stages are the tail of Status — there is no separate deploy column.

Triage = three decisions + an owner: type; severity (bugs); area(s); who owns it and which epic.

Severity: S1 wrong-number-shown-as-right / crash; S2 unusable feature; S3 display only; S4 cosmetic.

Sprint: two weeks, Monday start. Planning pulls Ready issues; grooming splits XLs; review verifies and retros.

Ready = typed, sized, owned, reproducible. **Done** = merged, tested, verified against the report.

Open-source facts: public board = world can watch, cannot edit. Interns get *project* Write without repository access. Change structure by editing `config/*.yml`, not the board.

OsdagBridge Project Management Handbook, version 1.2. Built from the SOPs and board configuration in `docs/pm/` and `config/`. Keep it next to the board.